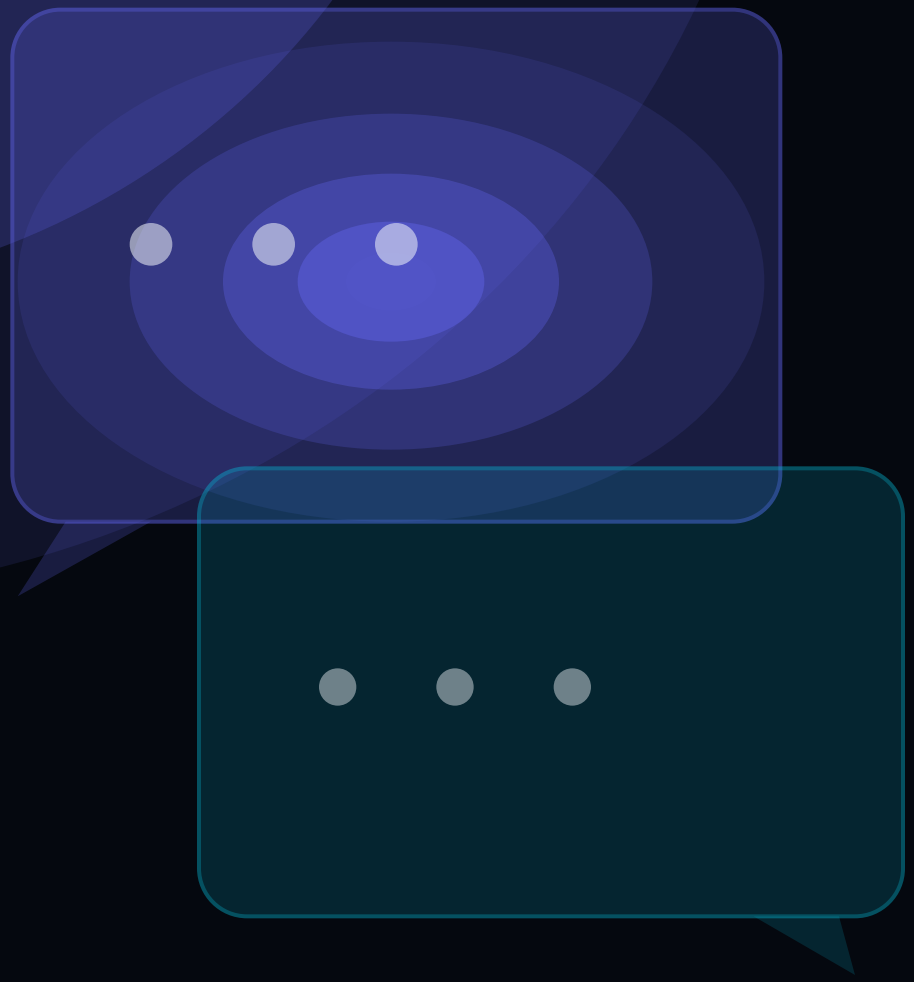


Les

Débatteurs

Plateforme de débat collaboratif pour entreprises — discussions, votes, propositions, groupes et rôles.

PYTHON FLASK SQLITE JS VANILLA

Soufiane B. · 2025

Problématique

Comment une entreprise peut-elle structurer ses échanges de façon transparente, traçable et démocratique ?

01

Décisions éparpillées

Les prises de décision se perdent dans des e-mails ou réunions sans trace. Impossible de retrouver l'origine d'un choix.

02

Aucun vote formel

Pas d'outil pour mesurer l'adhésion réelle. Les décisions reflètent trop souvent les voix les plus fortes, pas la majorité.

03

Périmètre incontrôlé

Tous les collaborateurs accèdent à tout. Pas de segmentation par équipe ni de droits différenciés selon le rôle.

Notre Solution

Application web centralisant les échanges d'une entreprise : créer, voter, commenter et proposer — le tout organisé par groupes et rôles. Déployable sans infrastructure externe.

Discussions structurées

01

Questions ouvertes (propositions classées) ou fermées (vote Pour/Contre). Priorité et clonage entre groupes.

Score composite original

02

Votes nets · sentiment pondéré · engagement logarithmique · bonus controverse · atténuation par profondeur.

Groupes & périmètres

03

Chaque discussion cible un groupe précis. On ne voit que les discussions de ses propres groupes.

Hiérarchie de rôles

04

CEO — Manager — Employé avec droits différenciés. Promotion et rétrogradation en temps réel.

Architecture Technique

FRONTEND

HTML5 · CSS3 · JavaScript Vanilla

Fetch API · Jinja2 Templates · DOM natif

BACKEND

Python 3.12 · Flask

Routes HTTP · Sessions · Logique métier

BASE DE DONNÉES

SQLite3 · 7 tables

Requêtes paramétrées · Migration auto

POURQUOI CES CHOIX ?

Flask

Micro-framework sans surcharge — chaque brique comprise sans magie cachée.

SQLite

Zéro config, fichier unique. Migration vers PostgreSQL en 5 lignes.

Vanilla JS

Fetch API native suffit. Pas de React = moins de dépendances.

Jinja2

SSR : HTML généré par Python, pas d'API REST à gérer séparément.

Modèle de Données · 7 Tables

Client

id · email · name · last_name · password · company · role

Discussions

id · id_c → Client · title · content · type · date · state · priority

Comments

id · id_c · id_d · content · type · avis · id_reply · state

Likes

id · id_c · target · id_target · like (+1/−1) · UNIQUE

Groups

id_group · name

GroupMembers

id · id_g → Groups · id_c → Client

DiscussionGroups

id_g → Groups · id_d → Discussions (many-to-many)

Toutes les relations : FOREIGN KEY. Requêtes via paramètres liés (?) — protection injection SQL.

Fonctionnalité Clé — Les Discussions

Question Ouverte

- Propositions de réponses soumises par les membres
- Chaque proposition : réponses Pour / Contre / Neutre
- Score composite classe les propositions par intérêt réel
- Toggle unique : switcher entre Proposition et Commentaire
- Imbrication jusqu'à 5 niveaux de profondeur
- Clôture possible une fois la décision actée



Question Fermée

- Vote binaire direct : Pour ou Contre la discussion
- Barre Pour/Contre animée en temps réel (Fetch API)
- Commentaires classés par popularité (net votes)
- Mode délibératif simple — pas de propositions
- Clonage vers un autre groupe avec filiation conservée
- Réouverture possible après clôture

Groupes & Système de Rôles

HIÉRARCHIE

CEO

Droits complets : membres, rôles, groupes, toutes les discussions

Manager

Créer discussions, gérer les membres de ses groupes

Employé

Commenter, voter, participer à ses groupes

CONCEPT DE GROUPES

- **Appartenance multiple**
  Un utilisateur peut rejoindre plusieurs groupes, dès l'inscription.
- **Périmètre strict**
  On ne voit que les discussions de ses groupes. Isolation totale.
- **Gestion CEO/Manager**
  Création et gestion des membres en temps réel. Ajout/retrait immédiat.
- **Clonage inter-groupes**
  Discussion dupliquée vers un autre groupe, filiation conservée.
- **Inscription**
  Rejoindre des groupes est proposé optionnellement à la création.

Algorithme de Score des Propositions

$$\text{Score} = V_{net} + \sum(\text{créd.} \times \text{dir.}) + \log(1+n)\times 2 + \sqrt{(\min(P,C))}\times 1.5 + \text{Profondeur}$$

V
Votes nets

`likes - dislikes`
Approbation directe sur la proposition. Poids de base du score.

?
Sentiment pondéré

`∑(crédibilité_réponse × direction)`
Pour/Contre pondérés par leurs propres likes. Qualité > quantité.

log
Engagement

`log(1 + votes + nb_réponses) × 2`
Croissance log — évite l'effet viral. 100 réponses ≠ 100× 1 réponse.

√
Controverse

`√(min(pour, contre)) × 1.5`
Bonus si les deux camps s'expriment. Débat intense = intérêt élevé.

+ Profondeur : sous-réponses atténuées par facteur 0.6^depth — les échanges profonds comptent, moins que les contributions directes.

Défis Techniques & Solutions

01	Commentaires imbriqués sans limite de profondeur	→	Récursif côté client + max depth adaptative (2/4/5 selon écran) → redirection vers /main_comment
02	Score CSS non mis à jour après vote AJAX	→	score.closest('.score-proposition') → update classes score-pos / score-neg / score-neutre
03	Colonne 'priority' absente de la BDD existante	→	migrate_db() au démarrage : PRAGMA table_info → ALTER TABLE conditionnel si colonne absente
04	Clonage de discussion avec filiation entre groupes	→	Colonne cloned_from + section 'Commentaires clonés' séparée + marqueur visuel

Bilan & Perspectives

CE QUE NOUS AVONS RÉALISÉ

- Application full-stack fonctionnelle et déployable
- Système de rôles CEO / Manager / Employé opérationnel
- Algorithme de score composite original
- Interface responsive et cohérente
- Cycle de vie complet des discussions
- Migration BDD automatique au démarrage

POUR ALLER PLUS LOIN

- Hashage des mots de passe (bcrypt)
- Notifications temps réel (WebSocket / SSE)
- API REST + application mobile
- Export PDF des discussions
- Dashboard analytique pour CEO
- Tests unitaires (pytest)

Merci de votre attention · Des questions ?